

UCS503 - Software Engineering

AgriGuard

AI-Based Crop Disease Detection and Advice for Farmers

Submitted by:

1024030114 Vaibhav Chaudhary

1024030155 Harshit Kasrija

1024030179 Harnoor

Group: XXXX BE Third Year, CSE

Submitted to:

Ms. Paramveer Kaur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	3
1.1	Background and Context	3
1.1.1	Problem Statement	3
1.2	Project Objectives	3
2	Methodology and Design	5
2.1	System Architecture	5
2.1.1	High-Level Design	5
2.1.2	Technical Stack	5
2.2	Implementation Details	6
2.2.1	Module 1: Disease Detection Model	6
2.2.2	Module 2: Severity Estimation and Treatment Advice	7
2.2.3	Module 3: Web Application and User Interface	7
2.2.4	Module 4: AgriBot Chatbot (Current Status)	10
3	Results and Evaluation	11
3.1	Testing Methodology	11
3.2	Performance Metrics	11
3.3	Analysis	13
4	Conclusion	15
4.1	Summary of Work	15
4.2	Key Findings	15
4.3	Future Work and Recommendations	15
4.4	Final Remarks	15
	Bibliography	17

Introduction

1.1 Background and Context

Crop diseases such as leaf blights, rusts and fungal infections spread quickly and can destroy a large part of a crop. They cause money loss for farmers and often lead to too much use of chemicals. At present, most farmers find out the disease using their own experience, or they wait for an agriculture expert to visit. This is slow, and a wrong or late decision can make the damage worse.

Computer vision and deep learning can help here. Earlier research has shown that a model trained on leaf photos can recognise many plant diseases [MHS16]. Our project, **AgriGuard**, uses this idea to build a simple web application. A farmer uploads a photo of an affected leaf and gets the disease name, how serious it is, and what to do next. The project also shows how a machine learning model can be connected with a website, a database, user login and testing to form one working system.

1.1.1 Problem Statement

Diagnosis of crop diseases is manual, slow and scattered. Farmers find it hard to know which disease is present, how severe it is and which treatment to use. There is a need for a fast, low-cost and easy tool that can identify the disease from a photo and explain the next steps in simple language, without depending on paid services.

1.2 Project Objectives

1. **Disease detection:** Identify the crop disease from a leaf photo using a deep learning model trained on the PlantVillage dataset, and measure its accuracy, precision and recall on a separate test set.
2. **Severity and confidence:** Show a confidence score and estimate the severity (Low, Medium or High) for every result.
3. **Advice:** Give simple disease information and treatment advice for each detected disease.
4. **Records and languages:** Save every report so that the farmer can see old results, and support English, Hindi and Punjabi in the interface.

5. **Free tools:** Build the complete system using only free and open-source tools, with a clear separation between the website, backend and model.

These objectives are specific and can be checked: the model results are measured on unseen test images, and the system is checked with test cases in Chapter 3.

Methodology and Design

2.1 System Architecture

AgriGuard follows a three-tier design: a website for the user, a backend that does the work, and a database for storage.

2.1.1 High-Level Design

The main parts of the system are:

- **Website (frontend):** Pages for login, scanning a crop, viewing results, report history and the AgriBot chat.
- **Backend:** Handles login, checks the uploaded image, runs the model, measures severity, prepares the advice and saves the report.
- **Database and storage:** Stores users, reports and chat messages. Uploaded images are kept as files and linked to their reports.

For one photo, the system works in these steps:

1. The farmer logs in and uploads or captures a leaf photo (JPG, PNG or WEBP, up to 8 MB).
2. The backend checks that the file is a proper image.
3. The MobileNetV2 model predicts the disease and gives a confidence score.
4. An OpenCV step measures how much of the leaf is damaged and gives a severity level.
5. The result and advice are shown to the farmer and the report is saved in the history.

2.1.2 Technical Stack

- **Framework/Language:** Python for the backend (FastAPI) and model training, JavaScript with React for the website.
- **Database:** SQLite.

- **Tools:** Kaggle notebook with a free GPU for training, Git and GitHub for source control, and an AI coding assistant (Antigravity) to help build the web application from our written specification. The team ran and checked the code.
- **Libraries:** PyTorch [PGML+19] and torchvision for the model, OpenCV and Pillow for images, scikit-learn and matplotlib for results, and Tailwind CSS for the interface design.

All tools are free and open source, and no paid service or external AI service is needed.

2.2 Implementation Details

2.2.1 Module 1: Disease Detection Model

Dataset. We used the PlantVillage dataset with color images [HS15]. Each folder is one class, named after the crop and the disease, for example `Tomato_Early_blight`. The copy we used was already divided into `train` and `val` folders. We kept 15% of the `train` folder (chosen with a fixed random seed) as a validation set to pick the best training round. The `val` folder of the dataset was not used during training and was used only once at the end as the test set. The dataset has 38 classes from 14 crops, and the application supports all of them.

Model. We used transfer learning. We started with MobileNetV2 [SHZ+18], which was already trained on ImageNet, and replaced its last layer so that it gives one output for each disease class. This model is small and fast, so it could be trained on a free GPU and can run on an ordinary computer. Training was done in two phases, as shown in Table 2.1.

Setting	Value
Base model	MobileNetV2 (ImageNet weights)
Image size, batch size	224×224 pixels, 64
Optimizer	AdamW
Phase A	2 epochs, only the new last layer trained (learning rate 10^{-3})
Phase B	6 epochs, last 6 blocks also trained (learning rate 10^{-4})
Augmentation	Random crop, horizontal and vertical flip, rotation up to 20 degrees, color change
Best model	Saved at the epoch with the best validation accuracy

Table 2.1: Training settings of the disease model

The best model and the list of class names are saved to files, and the application loads exactly these files to make predictions.

2.2.2 Module 2: Severity Estimation and Treatment Advice

Severity. The model gives only the disease name, so severity is measured separately with OpenCV. The leaf is separated from the background using color (HSV), the healthy green pixels are counted, and the remaining leaf pixels are counted as damaged. The damaged fraction is converted to a level: below 10% is Low, 10% to 30% is Medium, and above 30% is High. This is a simple color-based estimate and not a trained model. The application also shows an overlay on the photo, so the farmer can see which part was counted.

Advice. The advice comes from a prepared knowledge base with one entry for each disease class. It suggests simple and organic steps first, names chemicals only by their active ingredient, gives no exact dose, and tells the farmer to read the product label and ask a local agriculture officer. If the model confidence is low, the application asks for a clearer photo instead of giving advice.

2.2.3 Module 3: Web Application and User Interface

The website is made for non-technical users, with large buttons and a green theme. The login page (Figure 2.1) has one-click demo accounts for the two roles, Farmer and Expert, along with normal sign in and account creation. The language can be changed to English, Hindi or Punjabi from the top bar (Figure 2.2).

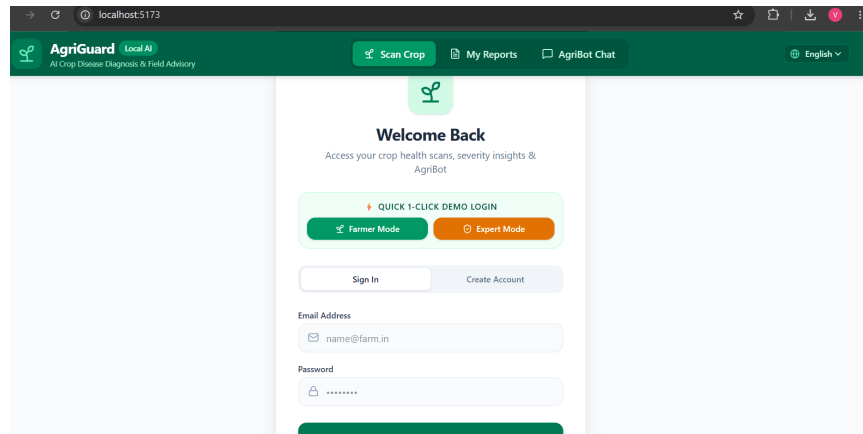


Figure 2.1: Login page with Farmer and Expert demo accounts

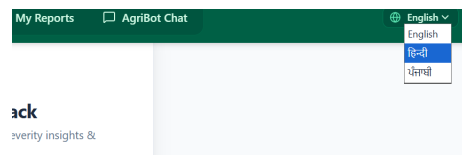


Figure 2.2: Language selector (English, Hindi, Punjabi)

On the Scan Crop page (Figure 2.3), the farmer can drag and drop a leaf photo, browse for one, or open the camera. A link explains which crops are supported and what the model cannot do. After a photo is chosen, a preview is shown (Figure 2.4) and the farmer can change or remove it.

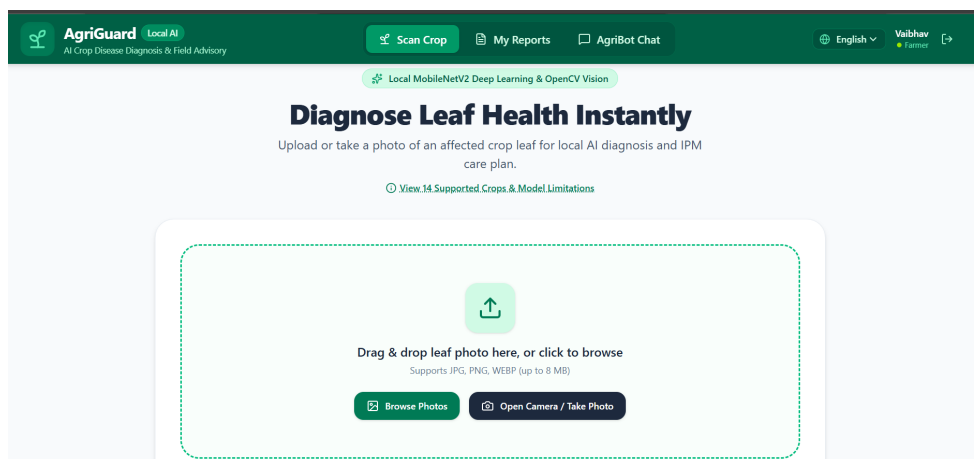


Figure 2.3: Scan Crop page for uploading or capturing a leaf photo

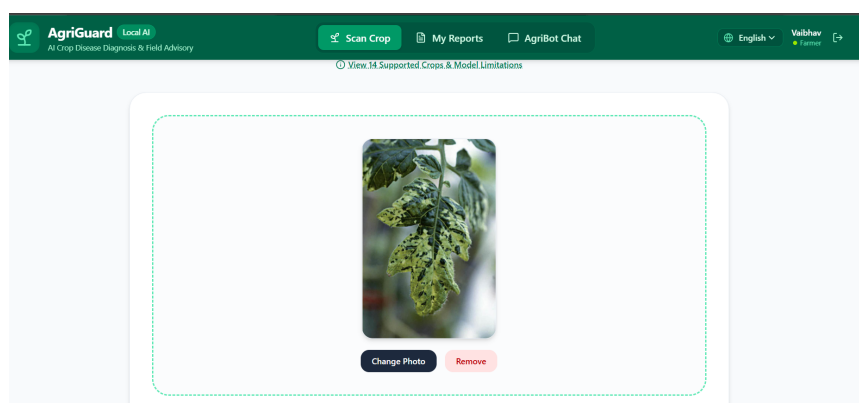


Figure 2.4: Photo preview before analysis

Below the upload box, the page shows tips for taking a good photo and optional boxes for crop, field location and notes (Figure 2.5). The analysis starts when the farmer presses *Analyze Leaf Now*.

The screenshot shows the lower part of the AgriGuard web application interface. It features a yellow box with 'Photo Quality Tips' including instructions on camera stability, lighting, and crop focus. Below this are three optional input fields: 'Crop (Optional)' with a dropdown menu showing '-- Auto-detect from image --', 'Farm / Field Location (Optional)' with a text input showing 'e.g. Field #3, North Plot', and 'Observations or Notes (Optional)' with a text input showing 'e.g. Observed yellow spots after recent rain...'. At the bottom is a large green button labeled 'Analyze Leaf Now'.

Figure 2.5: Photo tips, optional details and the Analyze Leaf Now button

The result page (Figure 2.6) shows the crop, the type of disease, the disease name with its scientific name, a short description, the severity with the percentage of damaged area, and the confidence of the model. An overlay marks the damaged parts of the leaf, and the farmer can switch back to the original photo. A button opens AgriBot for the same diagnosis.

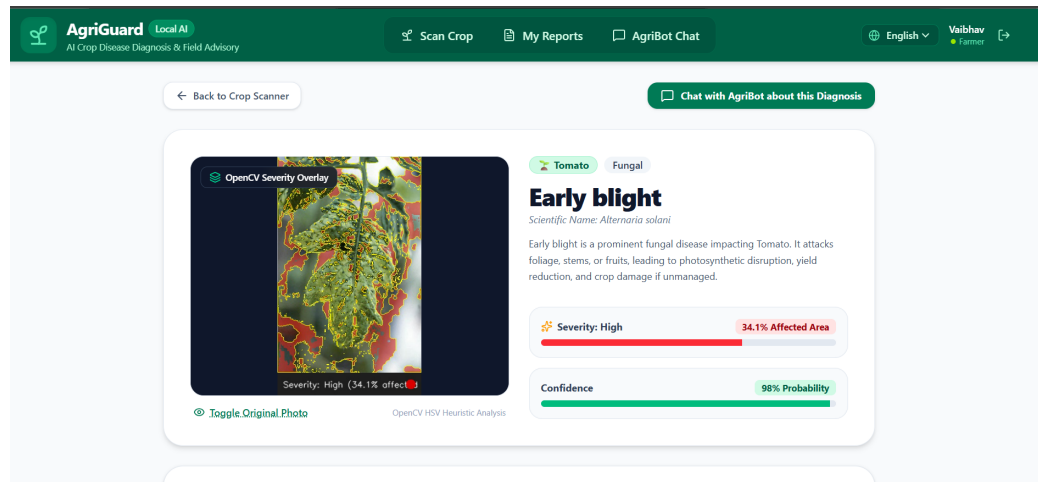


Figure 2.6: Result page for a tomato leaf: Early blight, High severity (34.1% damaged area), 98% confidence

2.2.4 Module 4: AgriBot Chatbot (Current Status)

AgriBot is meant to answer the farmer's questions about the detected disease and the action needed. The chat page, the button on the result page and the saving of chat history are in place. However, the chatbot is still a prototype: we have not trained or tested it properly yet, so its answers can be short or incomplete, and we have not measured how good it is. For this reason AgriBot is not included in the evaluation in Chapter 3. Improving it is the first item of future work.

Results and Evaluation

3.1 Testing Methodology

We checked the system in the following ways:

- **Model testing:** The trained model was tested on images from the held-out test set that it never saw during training. We measured accuracy, precision, recall and F1 score for each class.
- **Functional testing:** We tried the main steps of the application by hand: login, uploading photos, viewing the result, report history and language change.
- **Input checking:** We tried a blurry photo, a photo that is not a leaf, a wrong file type and a file larger than 8 MB, to see that the application gives a proper message.
- **Performance testing:** We noted the time the application takes to show a result after a photo is uploaded.

3.2 Performance Metrics

Table 3.1 shows the results of the model on the test set. The test set has 10,861 images from all 38 classes. These images were not used in training.

Item	Result
Number of classes	38 (14 crops)
Test images	50,861
Test accuracy	98.89%
Precision (macro average)	0.984
Recall (macro average)	0.984
F1 score (macro average)	0.984
F1 score (weighted average)	0.989

Table 3.1: Model results on the held-out test set

Figure 3.1 shows how the accuracy changed during training. In the first epoch the training accuracy was about 83% and the validation accuracy about 92%. By the last epoch both were close to 99%. The two lines stay very close to each other, so the model did not overfit badly.

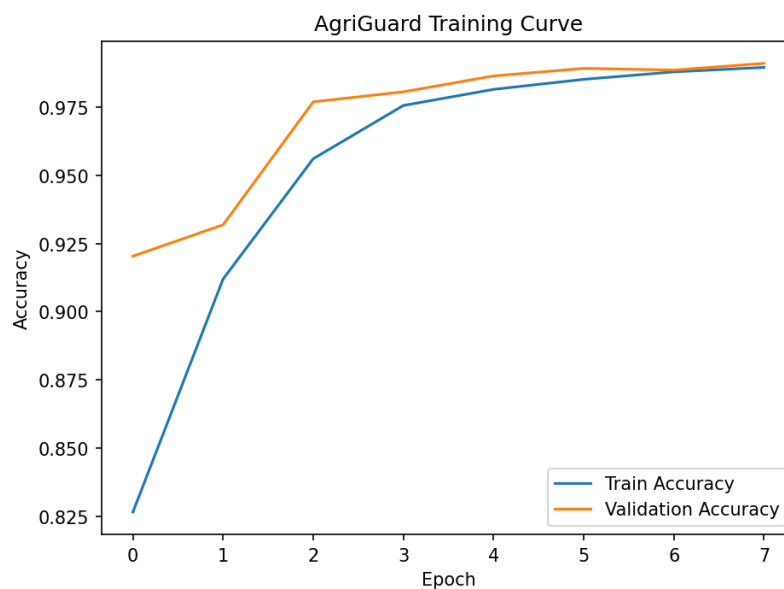


Figure 3.1: Training and validation accuracy during training

Out of the 38 classes, 30 have an F1 score of 0.98 or higher, and several classes reached a perfect score of 1.000. Table 3.2 lists the classes with the lowest scores.

Class	Precision	Recall	F1	Images
Corn, Cercospora / Gray leaf spot	0.922	0.806	0.860	103
Corn, Northern Leaf Blight	0.901	0.970	0.934	197
Tomato, Early blight	0.945	0.950	0.948	200
Potato, healthy	0.938	0.968	0.952	31
Tomato, Target Spot	0.989	0.947	0.967	281

Table 3.2: Classes with the lowest F1 score on the test set

Table 3.3 shows an example result from the running application (see also Figure 2.6).

Item	Output
Predicted disease	Tomato, Early blight (<i>Alternaria solani</i>), a fungal disease
Confidence	98%
Severity	High (34.1% of the leaf area damaged)

Table 3.3: Example output of the application

Table 3.4 lists the functional tests we carried out.

Test	What should happen	Result
Good leaf photo	Disease, confidence and severity are shown and the report is saved	[PASS]
Blurry or non-leaf photo	The application asks for a clearer photo	[PASS]
Wrong file type or file over 8 MB	The photo is rejected with a message	[PASS]
Login as Farmer and Expert	Correct pages for each role	[PASS]
Old reports	Past reports are listed after login	[PASS]
Language change	The interface changes to Hindi and Punjabi	[PASS]
Response time	Result shown within a few seconds	[PASS]

Table 3.4: Functional tests of the application

3.3 Analysis

- **Did we meet our objectives?** Yes, for the main parts. The model reached 98.89% accuracy on the test images. The main workflow works from start to end: a farmer uploads a leaf photo and gets the disease, confidence, severity and advice, and the report is saved. The interface supports three languages. The chatbot part of the advice objective is not finished yet.
- **Where did we do well?** A small pre-trained model gave very good results with a short training time on a free GPU, and the whole system runs on free tools.
- **What were the challenges?** The weakest results are for the corn leaf spot diseases (for example, a recall of 0.806 for Gray leaf spot), which can look alike, and for Tomato Early blight (F1 score 0.948). Early blight is also the disease in our example in Figure 2.6, so that result should be read with care. The PlantVillage photos are taken on plain backgrounds and many images show the same leaf, so the test accuracy is likely higher than what we would get on real farm photos. The severity is based on color, so light and background can change it. The model only knows the 38 classes it was trained on, so a leaf from another crop can get a wrong answer.
- **How did we handle them?** The application shows the supported crops, asks for a clearer photo when the confidence is low, and shows a warning that an

agriculture expert should check the treatment advice before it is used on a real farm.

Conclusion

4.1 Summary of Work

We built AgriGuard, a web application that finds crop diseases from leaf photos. We trained a MobileNetV2 model on the PlantVillage dataset, added an OpenCV step to estimate severity, prepared a knowledge base for advice, and built a multilingual website with login, report history and a chat page for AgriBot. The model reached 98.89% accuracy on the test images. Most of our objectives were met. The AgriBot chatbot is still a prototype.

4.2 Key Findings

- **Finding 1:** Transfer learning with a small model gave 98.89% accuracy on 10,861 unseen PlantVillage images with a short training time (see Table 3.1).
- **Finding 2:** A disease model alone is not enough for farmers. Severity, confidence, clear advice and saved reports make the result useful.
- **Finding 3:** Results on a clean dataset do not guarantee the same results on real farm photos, so testing on real photos is important.

4.3 Future Work and Recommendations

1. Finish and test AgriBot properly, using a checked knowledge base.
2. Train with real farm photos, add more crops, and reject photos that are not leaves.
3. Use a trained model for severity instead of the color method.
4. Complete the expert review feature, where agriculture officers can verify reports.
5. Make a mobile application and put the system online with a permanent database.

4.4 Final Remarks

This project taught us how to take a machine learning model from training to a working application, and how much the surrounding parts (image checks, user roles,

storage and testing) matter. With checked advice and real farm data, a tool like AgriGuard could become useful for farmers.

Bibliography

- [HS15] David P. Hughes and Marcel Salathé. “An open access repository of images on plant health to enable the development of mobile disease diagnostics”. In: *arXiv preprint arXiv:1511.08060* (2015) (cit. on p. 6).
- [MHS16] Sharada P. Mohanty, David P. Hughes, and Marcel Salathé. “Using Deep Learning for Image-Based Plant Disease Detection”. In: *Frontiers in Plant Science* 7 (2016), p. 1419 (cit. on p. 3).
- [PGML+19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2019 (cit. on p. 6).
- [SHZ+18] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. “MobileNetV2: Inverted Residuals and Linear Bottlenecks”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018 (cit. on p. 6).